

Use this cheat sheet for various packages/methods, their descriptions, and examples.

List

append()

The `append()` method is used to add an element to the end of a list.

Syntax:

```
list_name.append(element)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
fruits = ["apple", "banana", "orange"] fruits.append("mango") print(fruits)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



copy()

The `copy()` method is used to create a shallow copy of a list.

Example:

```
my_list = [1, 2, 3, 4, 5] new_list = my_list.copy() print(new_list) # Output: [1, 2, 3, 4, 5]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



count()

The count() method is used to count the number of occurrences of a specific element in a list in Python.

Example:

```
my_list = [1, 2, 2, 3, 4, 2, 5, 2] count = my_list.count(2) print(count) # Output: 4
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Creating a List

A list is a built-in data type that represents an ordered and mutable collection of elements. Lists are enclosed in square brackets `[]` and elements are separated by commas.

Example:

```
fruits = ["apple", "banana", "orange", "mango"]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



del

The del statement is used to remove an element from list. It removes the element at the specified index.

Example:

```
my_list = [10, 20, 30, 40, 50] del my_list[2] # Removes the element at index 2 print(my_list)  
# Output: [10, 20, 40, 50]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



extend()

The `extend()` method is used to add multiple elements to a list. It takes an iterable (such as another list, tuple, or string) and appends each element of the iterable to the original list.

Syntax:

```
list_name.extend(iterable)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
fruits = ["apple", "banana", "orange"] more_fruits = ["mango", "grape"] fruits.extend(more_fruits) print(fruits)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Indexing

Indexing in a list allows you to access individual elements by their position. In Python, indexing starts from 0 for the first element and goes up to 'length_of_list - 1'.

Example:

```
my_list = [10, 20, 30, 40, 50] print(my_list[0]) # Output: 10 (accessing the first element) print(my_list[-1]) # Output: 50 (accessing the last element using negative indexing)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



insert()

The `insert()` method is used to insert an element.

Syntax:

```
list_name.insert(index, element)
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

**Example:**

```
my_list = [1, 2, 3, 4, 5] my_list.insert(2, 6) print(my_list)
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Modifying a List

You can use indexing to modify or assign new values to specific elements in the list.

Example:

```
my_list = [10, 20, 30, 40, 50] my_list[1] = 25 # Modifying the second element print(my_list)  
# Output: [10, 25, 30, 40, 50]
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



pop()

The pop() method is another way to remove an element from a list in Python. It removes and returns the element at the specified index. If you don't provide an index to the pop() method, it will remove and return the last element of the list by default

Example 1:

```
my_list = [10, 20, 30, 40, 50] removed_element = my_list.pop(2) # Removes and returns the element at index 2
print(removed_element) # Output: 30
print(my_list) # Output: [10, 20, 40, 50]
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

**Example 2:**

```
my_list = [10, 20, 30, 40, 50] removed_element = my_list.pop() # Removes and returns the last element
print(removed_element) # Output: 50
print(my_list) # Output: [10, 20, 30, 40]
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



remove()

To remove an element from a list. The remove() method removes the first occurrence of the specified value.

Example:

```
my_list = [10, 20, 30, 40, 50] my_list.remove(30) # Removes the element 30
print(my_list) # Output: [10, 20, 40, 50]
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



reverse()

The reverse() method is used to reverse the order of elements in a list

Example:

```
my_list = [1, 2, 3, 4, 5] my_list.reverse() print(my_list) # Output: [5, 4, 3, 2, 1]
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Slicing

You can use slicing to access a range of elements from a list.

Syntax:

```
list_name[start:end:step]
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

**Example:**

```
my_list = [1, 2, 3, 4, 5] print(my_list[1:4]) # Output: [2, 3, 4] (elements from index 1 to 3)
) print(my_list[:3]) # Output: [1, 2, 3] (elements from the beginning up to index 2) print(m
y_list[2:]) # Output: [3, 4, 5] (elements from index 2 to the end) print(my_list[::2]) # Out
put: [1, 3, 5] (every second element)
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



sort()

The sort() method is used to sort the elements of a list in ascending order. If you want to sort the list in descending order, you can pass the 'reverse=True' argument to the sort() method.

Example 1:

```
my_list = [5, 2, 8, 1, 9] my_list.sort() print(my_list) # Output: [1, 2, 5, 8, 9]
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example 2:

```
my_list = [5, 2, 8, 1, 9] my_list.sort(reverse=True) print(my_list) # Output: [9, 8, 5, 2, 1]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Tuple

count()

The count() method for a tuple is used to count how many times a specified element appears in the tuple.

Syntax:

```
tuple.count(value)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
fruits = ("apple", "banana", "apple", "orange")print(fruits.count("apple")) #Counts the number of times apple is found in tuple.#Output: 2
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



index()

The `index()` method in a tuple is used to find the first occurrence of a specified value and returns its position (index). If the value is not found, it raises a `ValueError`.

Syntax:

```
tuple.index(value)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
fruits = ("apple", "banana", "orange","apple")print(fruits.index("apple")) #Returns the index value at which apple is present.#Output: 0
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



sum()

The `sum()` function in Python can be used to calculate the sum of all elements in a tuple, provided that the elements are numeric (integers or floats).

Syntax:

```
sum(tuple)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
numbers = (10, 20, 5, 30)print(sum(numbers))#Output: 65
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



min() and max()

Find the smallest (min()) or largest (max()) element in a tuple.

Example:

```
numbers = (10, 20, 5, 30)print(min(numbers)) #Output: 5print(max(numbers))#Output: 30
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



len()

Get the number of elements in the tuple using len().

Syntax:

```
len(tuple)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
fruits = ("apple", "banana", "orange")print(len(fruits)) #Returns length of the tuple.#Output  
: 3
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

